

02 — Types

The fundamental types

Type	Typical size	Range / precision	Notes
bool	1 byte	true / false	One byte, not one bit
char	1 byte	-128..127 (usually)	A small integer that happens to print as text
int	4 bytes	about ±2.1 billion	The default integer
unsigned int	4 bytes	0 .. about 4.3 billion	No negatives
long long	8 bytes	about ±9.2 quintillion	When int is not enough
float	4 bytes	~7 decimal digits	The GPU workhorse
double	8 bytes	~15 decimal digits	Twice the bytes, twice the traffic
size_t	8 bytes (64-bit)	unsigned	The type of sizeof; used for sizes and indices

"Typical" is doing real work in that table. The C++ standard guarantees minimum sizes and relative ordering, not exact sizes. On the platforms in this lab the numbers above hold — but the correct way to know is to ask:

```
std::cout << sizeof(int) << "\n"; // prints 4 on this machine
```

sizeof is evaluated at compile time and yields the size in bytes.

Why this is a change from Python

A Python int grows without limit. 2**200 is fine. A C++ int is exactly 4 bytes of memory, always, and computing past its range wraps around silently:

```
int x = 2147483647; // the maximum
x = x + 1;          // now large and negative
```

No exception, no warning by default. This is the single most surprising thing about C++ arithmetic coming from Python.

A Python float is always what C++ calls a double. C++ makes you choose, and the choice costs bytes.

"~7 decimal digits" does not mean "10 million values"

Easy to conflate, and the distinction matters later.

How many values a `float` can represent: it has 32 bits, so at most $2^{32} \approx 4.29$ billion bit patterns. Subtracting the NaN and infinity encodings leaves roughly **4.28 billion finite values**. Far more than seven digits of counting.

What “~7 digits” actually describes is *relative* precision. A `float` has a 24-bit significand — 23 stored plus one implicit leading bit — and $2^{24} = 16,777,216 \approx 1.7 \times 10^7$. Take $\log_{10}$ and you get ≈ 7.2 . So: any decimal number written with 7 significant digits survives a round trip through a `float`.

The values are not evenly spaced. They are dense near zero and sparse far from it. Every power-of-two interval holds exactly the same *count* of floats — $2^{23} = 8,388,608$ — spread over an interval twice as wide as the previous one:

Interval	Floats in it	Gap between neighbours
[1, 2)	8,388,608	$\sim 1.2 \times 10^{-7}$
[1024, 2048)	8,388,608	$\sim 1.2 \times 10^{-4}$
$[2^{23}, 2^{24})$	8,388,608	1.0
$[2^{24}, 2^{25})$	8,388,608	2.0

That last row has a consequence worth internalizing:

```
float x = 16777216.0f;    // 2^24
x = x + 1.0f;
// x is STILL 16777216.0f
```

Above 2^{24} the gap between consecutive floats is 2, so adding 1 has nowhere to land and rounds straight back. **A `float` cannot count to 20 million one at a time.**

```
float sum = 0.0f;
for (int i = 0; i < 20000000; ++i) sum += 1.0f;
// sum stops at 16777216
```

`FLT_EPSILON` ($\approx 1.19 \times 10^{-7}$, or 2^{-23}) is the same fact stated near 1.0: the smallest value that, added to `1.0f`, produces something different from `1.0f`.

Exercise 09 measures this. `double` has a 53-bit significand — about 15–16 decimal digits — and the same structure, just with the wall much further out.

Why This Matters for CUDA

This is not a curiosity; it drives real design decisions in the kernels this lab builds:

- **Reduction kernels sum millions of elements.** Accumulating in `float` loses information exactly as above, which is why reductions often accumulate in FP32 even when the data is FP16, and sometimes in

FP64.

- **Tensor Cores multiply in FP16 and accumulate in FP32** for precisely this reason. Storage precision and accumulation precision are separate decisions.
- **Summation order changes the result.** A GPU reduction sums in a different order than a sequential CPU loop, so the two give different answers on identical input. That is why kernel correctness is checked with a *tolerance* and never with equality.
- **Pairwise and Kahan summation exist** to push this error down, at a cost in FLOPs or registers.

Declaration and initialization

```
int a;           // declared, value is GARBAGE -- whatever was in that memory
int b = 5;       // copy initialization
int c{5};        // brace initialization -- refuses narrowing conversions
auto d = 5;      // compiler deduces int
auto e = 5.0f;   // deduces float
```

An uninitialized local variable holds whatever bytes happened to be at that address. Not zero. Reading it is undefined behaviour, and the value may differ between runs, between optimization levels, and between compilers. This has no Python equivalent — Python has no concept of a variable that exists but holds garbage.

Prefer `int c{5}` when you want the compiler to reject a lossy conversion:

```
int x = 3.7;     // compiles, silently truncates to 3
int y{3.7};     // error: narrowing conversion
```

auto

`auto` asks the compiler to deduce the type. It is still static typing — the type is fixed at compile time, just not spelled out. This is nothing like a Python variable, which can be rebound to a different type at any moment.

Use `auto` when the type is obvious from the right-hand side or when spelling it out is noise. Avoid it when the reader needs to know whether something is a `float` or a `double`.

const

```
const int n = 100;
n = 5;      // compile error
```

`const` means the compiler rejects any attempt to modify it. This is stronger than Python's naming convention for constants, which is only a convention.

`const` also documents intent in function signatures — `const float* data` says "I will read this, not write it" — and that becomes important in Module 02.

signed vs unsigned

```
unsigned int u = 0;
u = u - 1;    // not -1; wraps to about 4.3 billion
```

Mixing the two is a classic bug source:

```
int n = 5;
unsigned int m = 10;
if (n - m < 0) { /* never true: the subtraction is unsigned */ }
```

Compile with `-Wall -Wextra` and the compiler warns about sign comparison mismatches. Listen to it.

Casts

```
int a = 7, b = 2;
double bad = a / b;           // 3.0 -- integer division happened
FIRST
double good = static_cast<double>(a) / b; // 3.5
```

Integer division truncates. The cast has to happen *before* the division, not after; assigning to a `double` afterwards is too late.

`static_cast<T>(x)` is the explicit, searchable form. C-style `(double)a` also works and you will see it constantly in CUDA code, which is closer to C in style.

Why This Matters for CUDA

- `float` versus `double` is a **bandwidth decision**. Most GPU deep learning runs in FP32 or narrower precisely because bytes moved is the limiting resource.
- Mixed precision — FP16, BF16 — is this same table with smaller entries and sharper trade-offs. Understanding that a type is a fixed byte count is the prerequisite for that whole topic.
- Integer overflow in index arithmetic is a real bug in real kernels. Indexing a tensor with more than 2.1 billion elements using `int` overflows and writes to memory it does not own.
- `sizeof(T)` is what pointer arithmetic multiplies by, on the CPU and on the GPU alike.